

Libft'i Bařtan Sona Gerekten Anlamak

.c dosyasından .o'ya, .o'dan libft.a'ya, Makefile'ın her satırına ve kütüphanedeki 43 fonksiyonun tek tek mantığına kadar her řeyi kendi yazdığım kodun üzerinden, örneklerle ve benzetmelerle anlattığım rehber.

Proje: Libft · **Kullanıcı:** ibcolak · **Kapsam:** Part 1 (23) + Part 2 (11) + Bonus (9) = 43 fonksiyon
Bu döküman kodu ezberletmek için deęil, **mantığını oturtmak** için hazırlandı.

İçindekiler

Sırayla oku, ya da direkt merak ettiğin bölüme atla.

Derleme Süreci: .c'den .a'ya Giden Yol	4
Kaynak kod (.c) derleyici için bir tarif, .o ise yarı hazır ürün	4
-Wall -Wextra -Werror ne işe yarıyor	4
Statik kütüphane (.a) ve ar komutu	4
Linkleme: her şeyin birleştiği son adım	5
Makefile: Bütün Bu İş Otomatikleştirilen Şey	6
Makefile aslında ne için var	6
Değişkenler: NAME, CC, CFLAGS, AR, RM	6
SRCS, BONUS_SRCS ve OBJS: .c listesinden .o listesine	7
Pattern rule: %.o: %.c libft.h	7
all, bonus, clean, fclean, re: hedefler	7
libft.h: Kütüphanenin Kimlik Kartı	8
Header guard: #ifndef / #define / #endif	8
Neden bu dört include: stddef, stdlib, string, unistd	8
Fonksiyon prototipleri: derleyiciye önceden verilen söz	9
t_list struct'ı: zincirli listenin tek bir halkası	9
Part 1: libc Fonksiyonları (23 Fonksiyon)	10
Part 2: Ekstra Fonksiyonlar (11 Fonksiyon)	23
Bonus: Zincirli Liste (Linked List) Fonksiyonları	33
test_libft.c: Bunları Nasıl Doğruladık	38
check() ve section(): küçük bir mini test altyapısı	38
Neden bazı testler C'nin kendi fonksiyonlarıyla karşılaştırıyor	38
Neden bazı fonksiyonlar görsel doğrulama istiyor	38
Sonuç: 61/61	38
Kapanış	39

Bu Döküman Ne İçin Var?

Libft'i baştan sona derleyip norm'a uydurup testlerden geçirdik. Ama "çalışıyor" ile "neden çalıştığını biliyorum" arasında büyük fark var. Bu döküman o farkı kapatmak için var. Amaç ezber değil: bir fonksiyonun koduna tekrar baktığında "bu satır neden burada" sorusuna kendi kendine cevap verebilmen.

Dört ana bölümden oluşuyor. Önce **derleme sürecini** anlatıyorum: bir .c dosyası nasıl çalıştırılabilir bir şeye dönüşüyor, .o dosyası tam olarak ne, kütüphane (.a) neden var. Sonra **Makefile'ı** satır satır açıyorum, çünkü make yazdığında arka planda olan biteni bilmeden onu sadece ezbere çalıştırmış olursun. Sonra **libft.h'ı**, yani kütüphanenin kimlik kartını. Ve en sonda, kütüphanedeki **43 fonksiyonun her birini** — ne işe yaradığını, mantığını adım adım, gerçek kodun üzerinden ve kısa örneklerle.

NASIL OKUMALI

Sırayla okumak en mantıklısı çünkü sonraki bölümler öncekilere yaslanıyor (örneğin fonksiyonları anlamadan önce .o ve Makefile mantığını bilmek işine yarar). Ama zaten çalışan kısmı atlayıp direkt merak ettiğin fonksiyona da gidebilirsin, her fonksiyon kendi başına da anlaşılır şekilde yazıldı.

Derleme Süreci: .c'den .a'ya Giden Yol

Elinde 34 tane `ft_isalpha.c`, `ft_strlen.c` gibi dosya var. Bunlardan tek bir `libft.a` dosyası çıkıyor. Aradaki yolculukta neler oluyor, onu anlatıyorum.

Kaynak kod (.c) derleyici için bir tarif, .o ise yarı hazır ürün

`gcc` bir `.c` dosyasını okuduğunda önce senin yazdığın C kodunu işlemciye anlamlı gelecek makine diline çevirir. Bu çeviriye **derleme** denir. Ama bu çıktı henüz çalıştırılabilir bir program değildir; ona **object dosyası** (`.o`) denir. İçinde makine kodu vardır ama eksik parçalar da vardır: mesela `ft_strlen.c` içinde `ft_isalpha` çağırıyordun, derleyici "böyle bir fonksiyon var" diye günceller ama gerçek adresini bilmez, çünkü o başka bir dosyada. O adresi bulup boşlukları doldurma işi sonraki adımda, **linker**'a kalır.

BENZETME

Mutfakta 34 farklı malzemeyi ayrı ayrı hazırladığını düşün: soğanı doğradın, eti pişirdin, sosu karıştırdın. Her biri kendi başına bir "yarı ürün" — henüz yemek değil ama işlenmiş. İşte `.o` dosyaları bunlar. Hepsini bir kaba koyup birleştirdiğinde (linkleme) ortaya yemek (çalıştırılabilir program ya da kütüphane) çıkıyor.

Peki neden direkt `.c`'den tek seferde çalıştırılabilir dosya üretmiyoruz da araya `.o` koyuyoruz? Çünkü sadece `ft_split.c`'de bir satır değiştirdiğinde, diğer 33 dosyayı tekrar derlemek zaman kaybı. Object dosyaları sayesinde sadece değişen dosya yeniden derlenir, geri kalanlar zaten hazır `.o` olarak durur. Bunu birazdan Makefile bölümünde `%.o: %.c libft.h` kuralında tam olarak nasıl sağladığımızı göreceksin.

-Wall -Wextra -Werror ne işe yarıyor

Bizim Makefile'imizde `CFLAGS = -Wall -Wextra -Werror` satırı var. Üçü de derleyiciye "gözünü dört aç" demenin farklı seviyeleri:

Flag	Ne yapar
<code>-Wall</code>	Kullanılmayan değişken, karşılaştırılamaz tipler gibi yaygın hataları uyarı olarak gösterir
<code>-Wextra</code>	-Wall'ın atladığı, daha az yaygın ama yine önemli uyarıları da ekler
<code>-Werror</code>	Bütün bu uyarıları hataya çevirir; biri bile varsa derleme durur

Bu projede baya işimize yaradı: `unistd.h` eksikken `write()` kullandığımızda, ya da `ft_strjoin` içinde bir pointer'ı `char`'a atarken derleyici bunları es geçmedi, direkt hata verdi. Sıkıcı gelebilir ama aslında küçük hataları çalışma anına kalmadan yakalatan bir güvenlik ağı.

Statik kütüphane (.a) ve ar komutu

34 tane `.o` dosyan var. Bunları her yeni projede tek tek elle linklemek yerine, hepsini **tek bir arşiv dosyasında** topluyoruz: `libft.a`. Buna **statik kütüphane** denir. "Statik" kelimesi önemli: kütüphanenin kodu, onu kullanan programın içine derleme anında gerçekten kopyalanır (dinamik kütüphanelerde `.so` gibi- bu iş çalışma anında olur).

BENZETME

`.a` dosyası bir alet kutusu gibi düşünülebilir. İçinde tornavida, çekiç, pense (yani `ft_strlen`, `ft_split`, `ft_atoi` ...) var, hepsi ayrı ayrı ama tek bir kutuda. Başka bir projede bu kutuyu (`-lft` ile) yanına aldığında içindeki aletlerden istediğini kullanabiliyorsun; linker sadece gerçekten çağırdığın fonksiyonları kutudan çekip alıyor.

Bu arşivi oluşturan komut `ar rcs libft.a $(OBJS)`. Harf harf: `r` dosyaları arşive ekle (zaten varsa güncelle), `c` arşiv yoksa sessizce oluştur, `s` arşivin içine bir "index" (bir çeşit içindekiler tablosu) yaz ki linker hangi fonksiyonun hangi `.o`'da olduğunu tek tek aramadan hızlıca bulsun.

Linkleme: her şeyin birleştiği son adım

Sen kendi projende (mesela push_swap'ta) `ft_split` çağırdığında, derleyici senin dosyanı `.o` 'ya çevirirken içine "burada `ft_split` diye bir fonksiyon çağırılıyor ama tanımı burada değil" diye bir not düşer. Derleme bitince **linker** devreye girer: senin `.o` dosyalarınla, gösterdiğin `libft.a` içindeki `.o` 'ları eşleştirir, eksik adresleri doldurur ve tek bir çalıştırılabilir dosya üretir. `-I yol/Libft -L yol/Libft -lft` satırındaki `-I` "prototipleri (libft.h) burada ara", `-L` "kütüphane dosyasını burada ara", `-lft` ise "libft.a'yı linkle" demek (`lib` ve `.a` kısımlarını otomatik tamamlıyor, sadece `ft` kısmını yazman yetiyor).

Makefile: Bütün Bu İş Otomatikleştiren Şey

`make` yazdığında arkada tam olarak ne çalışıyor, satır satır açıyorum. Aşağıdaki blok kendi Makefile'imizin kısaltılmış hâli (34 ve 9 dosyalık listeler uzun olduğu için üç nokta ile kısalttım, mantık aynı).

```
NAME = libft.a

CC = gcc
CFLAGS = -Wall -Wextra -Werror
AR = ar rcs
RM = rm -f

SRCS = ft_isalpha.c \
      ft_isdigit.c \
      ... (34 mandatory dosya) \
      ft_putnbr_fd.c

BONUS_SRCS = ft_lstnew.c \
             ... (9 bonus dosya) \
             ft_lstmap.c

OBJS = $(SRCS:.c=.o)
BONUS_OBJS = $(BONUS_SRCS:.c=.o)

all: $(NAME)

$(NAME): $(OBJS)
        $(AR) $(NAME) $(OBJS)

bonus: $(OBJS) $(BONUS_OBJS)
        $(AR) $(NAME) $(OBJS) $(BONUS_OBJS)

%.o: %.c libft.h
        $(CC) $(CFLAGS) -c $< -o $@

clean:
        $(RM) $(OBJS) $(BONUS_OBJS)

fclean: clean
        $(RM) $(NAME)

re: fclean all

.PHONY: all bonus clean fclean re
```

Makefile aslında ne için var

`make`, "şu dosyayı üretmek için şuna ihtiyacın var, o da şuna bağlı" şeklinde bir **bağımlılık haritası** okuyan bir araç. Sen elle `gcc -c ft_isalpha.c -o ft_isalpha.o` yazıp bunu 34 kere tekrarlamak yerine, tek kelime (`make`) yazıyorsun, o senin yerine hepsini sırayla yapıyor. Üstelik akıllı: bir dosya değişmediyse onu tekrar derlemez, zaman kazandırır.

BENZETME

Makefile bir yemek tarifi değil, daha çok bir **montaj hattı talimatı** gibi. "Önce şu parçayı hazırla, sonra bunu, hepsi hazır olunca birleştir" diyor. Sen sadece "montajı başlat" (`make`) diyorsun, hangi parçanın zaten hazır olduğunu, hangisinin yeniden yapılması gerektiğini o kendisi çözüyor.

Değişkenler: NAME, CC, CFLAGS, AR, RM

Değişken	Değeri	Ne işe yarar
NAME	libft.a	Üreteceğimiz kütüphanenin adı; her yerde tekrar "libft.a" yazmak yerine tek yerden değiştirilebiliyor

CC	gcc	Kullanılacak derleyici
CFLAGS	-Wall -Wextra -Werror	Derleyiciye geçilecek uyarı/hata bayrakları
AR	ar rcs	Arşiv (statik kütüphane) oluşturma komutu
RM	rm -f	Dosya silme komutu (-f: dosya yoksa hata verme, sessiz geç)

Bunları değişken yapmanın sebebi tekrar etmemek: yarın derleyiciyi `cc` olarak değiştirmek istersen tek satırı değiştiriyorsun, dosyanın her yerini taramana gerek kalmıyor.

SRCS, BONUS_SRCS ve OBJS: .c listesinden .o listesine

`SRCS` zorunlu kısmın 34 dosyasını, `BONUS_SRCS` ise linked list'in 9 dosyasını listeliyor. İkisini ayrı tutmamızın sebebi net: `make` (sadece zorunlu kısmı) ile `make bonus` (bonus dahil hepsini) arasında fark olması gerekiyordu.

`OBJS = $(SRCS:.c=.o)` satırı Make'in bir "yerine koyma" (substitution) özelliği: `SRCS` içindeki her elemanın sonundaki `.c` 'yi `.o` ile değiştirip yeni bir liste üretiyor. Yani `ft_isalpha.c` otomatik olarak `ft_isalpha.o` oluyor, tek tek yazmana gerek kalmıyor.

Pattern rule: %.o: %.c libft.h

Bu satır, "herhangi bir `X.o` dosyası üretmek istiyorsan, sana `X.c` ve `libft.h` lazım" demenin kısa yolu. `%` işareti joker karakter: 34 ayrı kural yazmak yerine (`ft_isalpha.o: ft_isalpha.c` , `ft_strlen.o: ft_strlen.c` , ...) tek satırda hepsini kapsıyoruz.

`libft.h`'i bağımlılığa eklemenin özel bir sebebi var: header dosyasında bir prototip değiştirirsen (mesela bir fonksiyonun parametresini), bütün `.c` dosyalarının o yeni haberle tekrar derlenmesi gerekir. Bu satır olmasaydı, sadece `.c` dosyası değişmediği için `make` o dosyaları "zaten güncel" sanıp atlayabilirdi -- header değişikliğini gözden geçirirdi.

Kuralın gövdesi: `$(CC) $(CFLAGS) -c $< -o $@`. `$<` ilk bağımlılık demek (yani `.c` dosyası), `$@` hedefin kendisi demek (yani üretilecek `.o`). Bu iki otomatik değişken sayesinde kural her dosya için otomatik olarak doğru isimlerle çalışıyor.

all, bonus, clean, fclean, re: hedefler

Hedef	Ne yapar
<code>make / make all</code>	Sadece zorunlu 34 dosyayı derleyip <code>libft.a</code> 'yı oluşturur
<code>make bonus</code>	Zorunlu + bonus (9 linked list) dosyalarını derleyip hepsini <code>libft.a</code> 'ya ekler
<code>make clean</code>	Sadece ara <code>.o</code> dosyalarını siler, <code>libft.a</code> 'ya dokunmaz
<code>make fclean</code>	<code>clean</code> 'i çalıştırır, sonra <code>libft.a</code> 'yı da siler
<code>make re</code>	Önce <code>fclean</code> , sonra <code>all</code> : sıfırdan temiz baştan derleme

En alttaki `.PHONY: all bonus clean fclean re` satırı ise ince bir detay: Make normalde bir hedef adında dosya varsa "zaten hazır" deyip o kuralı atlar. Diyelim projede yanlışlıkla `clean` adında bir dosya oluştu -- `.PHONY` olmasaydı `make clean` hiçbir şey yapmadan çıkabilirdi, çünkü "clean" isminde bir dosya zaten var sanırdı. `.PHONY` bu isimlerin gerçek dosya değil, sadece komut grupları (isim) olduğunu Make'e açıkça söylüyor.

libft.h: Kütüphanenin Kimlik Kartı

Her `.c` dosyasının en üstünde `#include "libft.h"` görüyorsun. Bu dosya olmadan hiçbir şey derlenmez -- neden bu kadar merkezi olduğunu anlatıyorum.

```
#ifndef LIBFT_H
# define LIBFT_H

# include <stddef.h>
# include <stdlib.h>
# include <string.h>
# include <unistd.h>

typedef struct s_list
{
    void                *content;
    struct s_list       *next;
} t_list;

int                ft_isalpha(int c);
int                ft_isdigit(int c);
...
char              *ft_strdup(const char *s1);
...
t_list             *ft_lstmap(t_list *lst, void *(*f)(void *), void (*del)(void *));

#endif
```

Header guard: #ifndef / #define / #endif

Bir projede `libft.h`'i birden fazla dosya include edebilir, hatta aynı dosya zincirleme include'lar yüzünden iki kere include edilebilir. Eğer önlem almazsan, derleyici aynı `struct s_list` tanımını iki kere görür ve "yeniden tanımlama" hatası verir.

BENZETME

Bunu bir kapiya "buradan bir kere geçtim" notu asmak gibi düşün. İlk giren `LIBFT_H` diye bir bayrak (`#define`) asıyor. İkinci kez aynı kapiya gelen biri `#ifndef LIBFT_H` ile "bu bayrak zaten asılı mı" diye bakıyor, asılıysa içeri hiç girmiyor (dosyanın geri kalanı atlanıyor). Böylece dosyanın içeriği yanlışlıkla iki kez işlenmiyor.

Neden bu dört include: stddef, stdlib, string, unistd

Header	Neden lazım
<code><stddef.h></code>	<code>size_t</code> ve <code>NULL</code> tanımları için (<code>ft_strlen</code> , <code>ft_memcpy</code> gibi fonksiyonların dönüş/parametre tipi)
<code><stdlib.h></code>	<code>malloc</code> , <code>free</code> için (<code>ft_calloc</code> , <code>ft_strdup</code> , <code>ft_split</code> , <code>ft_lstnew</code> gibi bellek ayıran her fonksiyon kullanıyor)
<code><string.h></code>	Norm gereği kendi fonksiyonlarımızı yazsak da, bazı yardımcı tipler ve derleyicinin bazı fonksiyonları "builtin" tanınması için gerekli olabiliyor
<code><unistd.h></code>	<code>write()</code> sistem çağrısı için (<code>ft_putchar_fd</code> , <code>ft_putstr_fd</code> , <code>ft_putendl_fd</code> , <code>ft_putnbr_fd</code> bunu kullanıyor)

DIKKAT

Bu projede `<unistd.h>` eksikti ve `-Werror` yüzünden derleme tamamen duruyordu: derleyici `write()`'i tanımadan kullanıldığını görünce "implicit declaration" uyarısını hataya çevirip derlemeyi kesiyordu. Tek satır eklemek dört dosyanın derlenmesini kurtardı.

Fonksiyon prototipleri: derleyiciye önceden verilen söz

Her satır (`int ft_isalpha(int c);` gibi) bir **prototip**: fonksiyonun adı, aldığı parametreler ve döndürdüğü tip. Prototip olmadan, mesela `ft_split.c` içinde `ft_strlen` çağırdığında derleyici bu ismi hiç tanımaz ve hata verir -- çünkü `ft_strlen`'in gerçek tanımı başka bir dosyada (`ft_strlen.c`), derleyici `ft_split.c`'yi derlerken o dosyayı görmüyor bile. Prototip, derleyiciye "böyle bir fonksiyon başka bir yerde var, bana güven, linkleme aşamasında bulunacak" demenin yolu.

Tab hizalaması (`int ft_isalpha , size_t ft_strlen`) norm'un istediği bir okunabilirlik kuralı: bütün fonksiyon isimleri aynı sütunda başlasın diye, tip adı ne kadar kısaysa o kadar fazla tab kullanılıyor.

t_list struct'ı: zincirli listenin tek bir halkası

BENZETME

Bir tren düşün. Her vagonun içinde yük var (`content`) ve bir sonraki vagona bağlanan bir kanca var (`next`). Vagonlar birbirine art arda bağlı, ama her vagon sadece **kendinden sonrakini** biliyor, öncekini değil. İşte `t_list` tam olarak bu: her `t_list` düğümü kendi verisini (`void *content` -- her tür veriyi tutabilsin diye `void*`) ve bir sonraki düğümün adresini (`struct s_list *next`) tutuyor. Son vagonun kancası boş (`next == NULL`) -- trenin bittiğini o gösteriyor.

Bonus bölümündeki 9 fonksiyonun hepsi bu tek yapı etrafında dönüyor: yeni vagon eklemek, saymak, sona gitmek, silmek, her vagonun içeriğine bir işlem uygulamak... Bonus bölümünü okurken bu treni aklında tut, hepsi çok daha kolay oturacak.

Part 1: libc Fonksiyonları (23 Fonksiyon)

Bunlar C'nin standart kütüphanesinde (`<ctype.h>`, `<string.h>`, `<stdlib.h>`) zaten var olan fonksiyonların kendi elimizle yazdığımız versiyonları. Amaç tekerleği yeniden icat etmek değil, bu fonksiyonların **içeride ne yaptığını** gerçekten anlamak -- ki ileride kendi karmaşık fonksiyonlarını yazarken aynı mantığı (döngü, pointer aritmetiği, sınır kontrolü) rahatça kurabilesin.

KARAKTER KONTROL / DONUSUM FONKSIYONLARI

Bu yedi fonksiyonun hepsi aynı kalıpta: bir karakteri (`int c` olarak alınıyor, çünkü `EOF` gibi char aralığının dışındaki değerleri de karşılayabilsin diye) bir aralıkla karşılaştırıp `1 / 0` ya da dönüştürülmüş karakteri döndürüyor. ASCII tablosunu bilmek burada işini kolaylaştırıyor: `'A' - 'Z'` 65-90 arası, `'a' - 'z'` 97-122 arası, `'0' - '9'` 48-57 arası.

`ft_isalpha` (`int c`)

Karakter bir harf mi (a-z ya da A-Z) diye bakar.

İki aralığı `||` ile birleştirip tek satırda kontrol ediyor: büyük harf aralığında mı, ya da küçük harf aralığında mı.

```
int ft_isalpha(int c)
{
    if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z'))
        return (1);
    return (0);
}
```

ORNEK

```
ft_isalpha('k'); // 1
ft_isalpha('9'); // 0
```

`ft_isdigit` (`int c`)

Karakter bir rakam mı (0-9) diye bakar.

`'0'` ile `'9'` arasında mı diye tek bir aralık kontrolü yapıyor.

```
int ft_isdigit(int c)
{
    if (c >= '0' && c <= '9')
        return (1);
    return (0);
}
```

ft_isalnum (int c)

Karakter harf ya da rakam mı diye bakar (alpha + numeric).

Kendi yazdığımız iki fonksiyonu -- `ft_isalpha` ve `ft_isdigit` -- birleştiriyor. Kod tekrarı yerine var olanı tekrar kullanmak (code reuse) burada net görülüyor.

```
int ft_isalnum(int c)
{
    if (ft_isalpha(c) || ft_isdigit(c))
        return (1);
    return (0);
}
```

ft_isascii (int c)

Karakter ASCII aralığında mı (0-127) diye bakar.

ASCII 7 bit ile temsil edilir, yani 0'dan 127'ye kadar 128 değer alır. Bu aralığın dışına çıkan (örneğin -1 ya da 200) bir değer ASCII değildir.

```
int ft_isascii(int c)
{
    if (c >= 0 && c <= 127)
        return (1);
    return (0);
}
```

ft_isprint (int c)

Karakter ekrana yazdırılabilir mi diye bakar (boşluk dahil, yeni satır hariç).

32 (boşluk karakteri) ile 126 (~) arası "görünür" ASCII karakterleridir. `\n`, `\t` gibi kontrol karakterleri bu aralığın dışında kaldığı için yazdırılabilir sayılmaz.

```
int ft_isprint(int c)
{
    if (c >= 32 && c <= 126)
        return (1);
    return (0);
}
```

ORNEK

```
ft_isprint('A'); // 1
ft_isprint('\n'); // 0
```

ft_toupper (int c)

Küçük harfi büyük harfe çevirir, değilse dokunmaz.

ASCII tablosunda 'a' ile 'A' arasında tam 32 birim fark var ($97 - 65 = 32$). Küçük harfe 32 çıkarınca büyük harf karşılığına düşüyorsun. Küçük harf değilse (zaten büyük harf, rakam, boşluk vs.) hiç dokunmadan aynen geri döndürüyoruz.

```
int ft_toupper(int c)
{
    if (c >= 'a' && c <= 'z')
        return (c - 32);
    return (c);
}
```

ORNEK

```
ft_toupper('k'); // 'K'
ft_toupper('5'); // '5' (degismedi)
```

ft_tolower (int c)

Büyük harfi küçük harfe çevirir, değilse dokunmaz.

ft_toupper 'ın tam tersi: büyük harfse 32 ekleyip küçük harf karşılığına geçiyor.

```
int ft_tolower(int c)
{
    if (c >= 'A' && c <= 'Z')
        return (c + 32);
    return (c);
}
```

BELLEK (MEMORY) FONKSIYONLARI

Bu fonksiyonlar tipten bağımsız çalışıyor -- `void *` alıp içeride `unsigned char *` 'a çeviriyorlar, çünkü bellekte byte byte gezinmek için en küçük, işaretsiz birim `unsigned char` (1 byte, 0-255 arası). Bu sayede bir `int` dizisinde de, bir `struct` 'ta da, bir string'de de aynı fonksiyonlar çalışabiliyor.

ft_memset (void *b, int c, size_t len)

Bir bellek alanının len byte'ını verilen değerle (c) doldurur.

b 'yi unsigned char* 'a çevirip, c 'yi de tek byte'a indirgeyip ((unsigned char)c), len kere art arda o değeri yazıyor. *ptr++ = value satırı hem şu anki adrese yazıyor hem pointer'ı bir sonraki byte'a kaydırıyor -- aynı satırda iki iş.

```
void      *ft_memset(void *b, int c, size_t len)
{
    unsigned char    *ptr;
    unsigned char    value;

    ptr = (unsigned char *)b;
    value = (unsigned char)c;
    while (len--)
    {
        *ptr++ = value;
    }
    return (b);
}
```

ORNEK

```
char buf[5];
ft_memset(buf, 'X', 5); // buf = "XXXXX"
```

ft_bzero (void *s, size_t n)

Bir bellek alanının n byte'ını sıfırlar.

Kendi başına hiçbir iş yapmıyor, direkt ft_memset(s, 0, n) 'ı çağırıyor. Yeni bir fonksiyon değil, ft_memset 'in özel bir kullanım şekli -- "her zaman sıfırla" ihtiyacı o kadar sık ki ayrı bir isim verilmiş.

```
void      ft_bzero(void *s, size_t n)
{
    ft_memset(s, 0, n);
}
```

BENZETME Bir beyaz tahtayı düşün. ft_memset "şu renkle boy" diyen genel fonksiyon, ft_bzero ise sadece "sil" (yani rengi 0'a boy) diyen kısayol.

ft_memcpy (void *dst, const void *src, size_t n)

src'deki n byte'ı dst'ye kopyalar (alanlar üst üste binmiyorsa güvenli).

Byte byte, `*d++ = *s++` ile hem okuyup hem yazıp iki pointer'ı da ilerletiyor. Norm subject'inde belirtilen özel bir durum var: `dst` ve `src` ikisi de `NULL` ise çökmeden `NULL` döndürmesi gerekiyor, o yüzden döngüden önce bu durumu ayrıca kontrol ediyoruz.

```
void      *ft_memcpy(void *dst, const void *src, size_t n)
{
    unsigned char      *d;
    const unsigned char *s;

    d = (unsigned char *)dst;
    s = (const unsigned char *)src;
    if (!dst && !src)
        return (NULL);
    while (n--)
    {
        *d++ = *s++;
    }
    return (dst);
}
```

DIKKAT Alanlar üst üste biniyorsa (overlap) bu fonksiyon güvenli değil -- baştan yazdığın byte'lar henüz okumadığın kaynağı ezebilir. O durumda `ft_memmove` kullanılmalı, aşağıda anlatıyorum.

ft_memmove (void *dest, const void *src, size_t n)

ft_memcpy gibi ama kaynak ve hedef üst üste binse bile güvenli çalışır.

Buradaki mantık projedeki en "ince" mantıklardan biri, adım adım izleyelim. İki durum var:

1. **dest kaynaktan önceyse** (`d < s`): baştan sona doğru kopyalamak güvenli, çünkü yazdığın byte, henüz okumadığın bir byte'ın üzerine gelmiyor.
2. **dest kaynaktan sonraysa**: baştan kopyalarsan, ileride okuyacağın kaynak byte'ları daha okumadan üzerine yazmış olursun. Çözüm: **sondan başa** doğru kopyalamak. Kod bunu `while (n--)` döngüsünün içinde `*(d + n) = *(s + n)` ile yapıyor -- `n` her turda 1 azaldığı için, aslında dizinin son elemanından ilk elemanına doğru gidiyorsun, sıra tam tersine dönmüş oluyor.

```
void      *ft_memmove(void *dest, const void *src, size_t n)
{
    unsigned char      *d;
    const unsigned char *s;

    d = (unsigned char *)dest;
    s = (const unsigned char *)src;
    while (n--)
    {
        if (d < s)
            *(d++) = *(s++);
        else
            *(d + n) = *(s + n);
    }
    return (dest);
}
```

BENZETME Aynı rafta yan yana duran kutuları bir kaydirdiğini düşün. Kutuları soldan sağa kaydırıyorsan, sağdaki boş yerden başlayıp geriye doğru taşırsın -- yoksa bir kutuyu taşımadan üzerine başka bir kutu koyup kaybedersin. `ft_memmove` tam olarak bu sağduyuyu koda döküyor.

ft_memchr (const void *s, int c, size_t n)

Bellek alanında n byte içinde c değerini arar, bulursa pointer döner.

Bir string fonksiyonu değil -- `'\0'` 'da durmaz, tam olarak `n` byte kadar arar. Bulduğu anda o adresi döndürüyor, bulamazsa `NULL`.

```
void      *ft_memchr(const void *s, int c, size_t n)
{
    unsigned char      *ptr;
    unsigned char      ch;

    ptr = (unsigned char *)s;
    ch = (unsigned char)c;
    while (n--)
    {
        if (*ptr == ch)
            return (ptr);
        ptr++;
    }
    return (NULL);
}
```

ft_memcmp (const void *s1, const void *s2, size_t n)

İki bellek alanını n byte boyunca karşılaştırır.

İlk farklı byte'ı bulduğunda, iki byte'ın farkını (*p1 - *p2) döndürüyor -- pozitifse s1 büyük, negatifse s2 büyük demek. Fark yoksa n byte boyunca döngü bitip 0 dönüyor (eşit demek).

```
int ft_memcmp(const void *s1, const void *s2, size_t n)
{
    const unsigned char *p1;
    const unsigned char *p2;

    p1 = (const unsigned char *)s1;
    p2 = (const unsigned char *)s2;
    while (n--)
    {
        if (*p1 != *p2)
            return (*p1 - *p2);
        p1++;
        p2++;
    }
    return (0);
}
```

STRING (KARAKTER DİZİSİ) FONKSİYONLARI

C'de string diye ayrı bir tip yok -- string dediğimiz şey aslında sonu '\0' (null terminator) ile biten bir char dizisi. Bu fonksiyonların çoğu bu '\0' 'ı arayarak ya da ona kadar sayarak çalışıyor.

ft_strlen (const char *s)

String'in uzunluğunu (null terminator hariç) döner.

'\0' 'a çarpana kadar sayacı artırıyor. Bu fonksiyon aslında kütüphanenin en çok kullanılan yardımcısı -- neredeyse her fonksiyon (strjoin, substr, strdup, split...) içeride buna yaslanıyor.

```
size_t ft_strlen(const char *s)
{
    size_t len;

    len = 0;
    while (s[len] != '\0')
        len++;
    return (len);
}
```

ft_strlcpy (char *dst, const char *src, size_t size)

src'yi dst'ye kopyalar ama dst'nin boyutunu (size) asla aşmaz.

Normal `strcpy`'nin tehlikesi, hedef buffer'ın boyutunu hiç sormaması -- taşarsa (buffer overflow) çöker. `strlcpy` bunu önüyor: en fazla `size - 1` karakter kopyalar, sona her zaman `'\0'` koyar. Dönüş değeri ilginç: her zaman `src`'nin **gerçek** uzunluğunu döner (kırpılmış olsa bile). Böylece çağırıcı taraf `dönüş >= size` kontrolüyle "kırpma oldu mu" diye anlayabiliyor.

```
size_t      ft_strlcpy(char *dst, const char *src, size_t size)
{
    size_t      src_len;
    size_t      i;

    if (dst == NULL || src == NULL)
        return (0);
    src_len = ft_strlen(src);
    if (size == 0)
        return (src_len);
    i = 0;
    while (i < size - 1 && src[i])
    {
        dst[i] = src[i];
        i++;
    }
    dst[i] = '\0';
    return (src_len);
}
```

ORNEK

```
char buf[4];
ft_strlcpy(buf, "Merhaba", 4); // buf = "Mer", donus = 7
```

Cikti → donen deger (7), buf'in boyutundan (4) büyük -> kırpma olduğunu anlıyoruz

ft_strlcat (char *dst, const char *src, size_t size)

src'yi dst'nin sonuna ekler, ama toplam uzunluk size'ı geçemez.

Önce `dst` 'nin şu anki uzunluğunu buluyor. Eğer `size` zaten `dst_len` 'den küçük ya da eşitse -- yani verilen boyut, dst'nin şu anki içeriğine bile yetmiyorsa -- hiçbir şey eklemekten `size + src_len` döndürüyor (BSD standardının tanımladığı özel durum). Normal durumda ise `size - 1` sınırına kadar src'den karakter ekliyor, sonuna `'\0'` koyuyor ve `dst_len + src_len` döndürüyor -- yani "eğer sınır olmasaydı sonuç kaç karakter olurdu" bilgisini veriyor.

```
size_t      ft_strlcat(char *dst, const char *src, size_t size)
{
    size_t    dst_len;
    size_t    src_len;
    size_t    i;

    if (dst == NULL || src == NULL)
        return (0);
    dst_len = ft_strlen(dst);
    src_len = ft_strlen(src);
    if (size <= dst_len)
        return (size + src_len);
    i = 0;
    while (src[i] && (dst_len + i) < (size - 1))
    {
        dst[dst_len + i] = src[i];
        i++;
    }
    dst[dst_len + i] = '\0';
    return (dst_len + src_len);
}
```

DIKKAT Dönüş değerinin "gerçekleşen" değil "olması gereken" uzunluğu vermesi kafa karıştırabilir ama bu tasarım kasıtlı: fonksiyon çağırının kırpmı olup olmadığını anlamasını sağlıyor.

ft_strchr (const char *s, int c)

String içinde c karakterinin ilk geçtiği yeri bulur.

Soldan sağa tarıyor, eşleşen ilk karakterin adresini döndürüyor. Özel bir durum var: c olarak '\0' verilirse, string'in bitiş noktasının adresini döndürmesi gerekiyor (çünkü '\0' da string'in bir parçası sayılır, teknik olarak string'i sonlandıran karakter). O yüzden döngü bitince ayrıca bu durumu kontrol ediyoruz.

```
char *ft_strchr(const char *s, int c)
{
    while (*s)
    {
        if (*s == (char)c)
            return ((char *)s);
        s++;
    }
    if ((char)c == '\0')
        return ((char *)s);
    return (NULL);
}
```

ORNEK

```
ft_strchr("merhaba", 'r'); // "rhaba" adresini doner
```

ft_strrchr (const char *s, int c)

String içinde c karakterinin son geçtiği yeri bulur.

ft_strchr 'in tam tersi ama farklı bir teknikle: ilk gördüğünde direkt dönmüyor, her eşleşmede last değişkenini güncelleyip taramaya devam ediyor. String bitene kadar en son güncellenen last, doğal olarak son eşleşme oluyor. Hiç eşleşme olmazsa last hâlâ NULL, o da doğru cevap.

```
char *ft_strrchr(const char *s, int c)
{
    const char *last;

    last = NULL;
    while (*s)
    {
        if (*s == (char)c)
            last = s;
        s++;
    }
    if ((char)c == '\0')
        return ((char *)s);
    return ((char *)last);
}
```

ft_strncmp (const char *s1, const char *s2, size_t n)

İki string'i en fazla n karaktere kadar karşılaştırır.

Karakter karakter ilerliyor. Fark bulunca `unsigned char` 'a çevirip farkı döndürüyor (imzasız yapmak önemli, yoksa Türkçe olmayan yüksek byte değerli karakterlerde negatif/pozitif işaret hatası olabilir). İkisinden biri `'\0'` 'a ulaşırsa (string bittiye) daha ileri gitmeden karşılaştırmayı orada kesip `0` (eşit) dönüyor -- ikisi de aynı noktada bitmiş demektir zaten, üsteki `if` zaten farkı yakalamıştı.

```
int ft_strncmp(const char *s1, const char *s2, size_t n)
{
    size_t i;

    i = 0;
    while (i < n)
    {
        if (s1[i] != s2[i])
            return ((unsigned char)s1[i] - (unsigned char)s2[i]);
        if (s1[i] == '\0' || s2[i] == '\0')
            return (0);
        i++;
    }
    return (0);
}
```

ft_strnstr (const char *big, const char *little, size_t len)

big içinde, ilk len karakter sınırı dahilinde, little alt string'ini arar.

Önce özel durum: `little` boşsa (`""`), her string'in içinde boş string "vardır" kabul edilir, direkt `big` 'i döndürüyoruz. Sonra `big` üzerinde geziniyoruz; `little` 'ın ilk harfiyle eşleşen her noktada, içteki `while` ile `little` 'ın tamamının eşleşip eşleşmediğine bakıyoruz -- hem `little` bitmiş olmalı (`little[j] == '\0'`) hem de `len` sınırını aşmamış olmalıyız.

```
char *ft_strnstr(const char *big, const char *little, size_t len)
{
    size_t i;
    size_t j;

    if (little[0] == '\0')
        return ((char *)big);
    i = 0;
    while (big[i] && i < len)
    {
        if (big[i] == little[0])
        {
            j = 0;
            while (little[j] && big[i + j] == little[j] && (i + j) < len)
                j++;
            if (little[j] == '\0')
                return ((char *)(big + i));
        }
        i++;
    }
    return (NULL);
}
```

ORNEK

```
ft_strnstr("hello world", "world", 11); // "world" adresini doner
ft_strnstr("hello world", "world", 7); // NULL (len sinirina takildi)
```

DONUSUM VE BELLEK AYIRMA (ALLOCATION) FONKSIYONLARI

ft_atoi (const char *nptr)

Bir sayı içeren string'i int'e çevirir ("ASCII to Integer").

Üç aşamalı çalışıyor: **(1)** baştaki boşluk/tab/yeni satır gibi karakterleri atla, **(2)** varsa **+** ya da **-** işaretini oku (art arda birden fazla **-** gelirse her biri işareti çeviriyor -- **sign *= -1**), **(3)** rakam kaldığı sürece sonucu **sonuc * 10 + yeni_rakam** ile büyötmeye devam et. Sonuç **long** tipinde tutuluyor, çünkü hesaplama sırasında geçici olarak **int** sınırlarını aşabilir; en sonda işaretle çarpılıp **int** 'e dönüyor.

```
int      ft_atoi(const char *nptr)
{
    int      sign;
    long     result;

    sign = 1;
    result = 0;
    if (nptr == NULL)
        return (0);
    while (*nptr == ' ' || (*nptr >= 9 && *nptr <= 13))
        nptr++;
    while (*nptr == '+' || *nptr == '-')
    {
        if (*nptr == '-')
            sign *= -1;
        nptr++;
    }
    while (*nptr >= '0' && *nptr <= '9')
    {
        result = result * 10 + (*nptr - '0');
        nptr++;
    }
    return (result * sign);
}
```

ORNEK

```
ft_atoi(" -42abc"); // -42 (bosluklari atlar, harfe gelince durur)
```

ft_calloc (size_t nmemb, size_t size)

nmemb elemanlık, her biri **size** byte'lık bir bellek alanı ayırır ve sıfırlar.

`malloc` 'tan farkı: ayırdığı belleği garanti olarak **sıfırlar** (çöp veri kalmaz). Önce çarpma taşması (overflow) kontrolü yapılıyor -- `nmemb * size` çok büyükse bu çarpma sistem belleğinin sınırını sarıp yanlışlıkla küçük bir sayıya dönebilir (integer overflow), bu da olması gerekenden çok daha küçük ve tehlikeli bir alan ayrılmasına yol açar. O yüzden çarpmadan önce "bu güvenli mi" diye kontrol ediliyor. Sonra `malloc` ile alan ayrılıp `ft_memset` ile sıfırlanıyor.

```
void      *ft_calloc(size_t nmemb, size_t size)
{
    void      *ptr;

    if (nmemb != 0 && size > (size_t) - 1 / nmemb)
        return (NULL);
    ptr = malloc(size * nmemb);
    if (!ptr)
        return (NULL);
    ft_memset(ptr, 0, size * nmemb);
    return (ptr);
}
```

BENZETME Boş bir defter aldığında sayfaların hepsi tertemiz olsun istersin, üzerinde eski birinin karalamaları kalsın. `malloc` sana rastgele kirlili sayfalar verebilir (önceden kullanılmış bellek), `calloc` ise sayfaları verirken üzerlerini de siler.

ft_strdup (const char *s)

Bir string'in heap üzerinde bağımsız bir kopyasını oluşturur.

`ft_strlen` ile uzunluğu bulup `+1` (null terminator için) kadar yer ayırıyor, sonra karakter karakter kopyalıyor. `start` değişkeni kopyalama sırasında `ptr` kaydırıldığı için (`*ptr++ = *s++`) dizinin başlangıç adresini kaybetmemek için tutuluyor -- döngü bitince `ptr` artık dizinin sonunu gösteriyor, dönmemiz gereken ise `start`.

```
char      *ft_strdup(const char *s)
{
    char      *ptr;
    char      *start;
    size_t    len;

    len = ft_strlen(s);
    ptr = (char *)malloc(len + 1);
    if (!ptr)
        return (NULL);
    start = ptr;
    while (len--)
        *ptr++ = *s++;
    *ptr = '\0';
    return (start);
}
```

Part 2: Ekstra Fonksiyonlar (11 Fonksiyon)

Bunlar standart C kütüphanesinde birebir karşılığı olmayan, 42 müfredatının kendi tanımladığı yardımcı fonksiyonlar. Çoğu, sonraki projelerde (push_swap, minishell, printf...) sürekli ihtiyaç duyacağın işlemler: string parçalama, birleştirme, sayı-string dönüşümü, fd'ye yazma.

ft_substr (char const *s, unsigned int start, size_t len)

s'in start indeksinden başlayarak en fazla len karakterlik bir alt string kopyasını döner.

Önce sınır kontrolleri yapıyor: `start` zaten string'in sonundan büyükse, kopyalanacak hiçbir şey yok demektir -- boş string (`ft_strdup("")`) dönülüyor, `NULL` değil (bu ayrım önemli, çünkü `ft_strtrim` gibi başka fonksiyonlar bu değeri sonra `ft_strlen` 'e güvenle geçirebilirsin diye). Sonra `len` , string'in geri kalanından uzunsa (`len > s_len - start`) otomatik olarak sınıra kadar kırılıyor. Böylece "len çok büyük verildi" diye çağırın tarafın endişelenmesine gerek kalmıyor.

```
char      *ft_substr(char const *s, unsigned int start, size_t len)
{
    size_t    i;
    size_t    s_len;
    char      *str;

    if (!s)
        return (NULL);
    s_len = ft_strlen(s);
    if (start >= s_len)
        return (ft_strdup(""));
    if (len > s_len - start)
        len = s_len - start;
    str = malloc(sizeof(char) * (len + 1));
    if (!str)
        return (NULL);

    i = 0;
    while (i < len && s[start + i] != '\0')
    {
        str[i] = s[start + i];
        i++;
    }
    str[i] = '\0';
    return (str);
}
```

ORNEK

```
ft_substr("Hello World", 6, 5); // "World"
ft_substr("Hello", 10, 5); // "" (start disariya tastigi icin)
```

ft_strjoin (char const *s1, char const *s2)

İki string'i art arda birleştirip yeni, tek bir string döner (ikisini de değiştirmez).

Toplam uzunluk kadar (s1_len + s2_len + 1) yer ayırıyor, sonra bir yardımcı fonksiyon -- ft_strjoin_copy -- ile önce s1'i, sonra doğru offsetten (collab + s1_len) başlayarak s2'yi kopyalıyor. Bu yardımcı fonksiyonun ayrı yazılmasının sebebi kod tekrarını önlemek: aynı "byte byte kopyala" işlemi iki kere gerekiyordu, tek fonksiyona çıkarıp iki kere çağırdık.

```
static void      ft_strjoin_copy(char *dst, const char *src, size_t len)
{
    size_t      i;

    i = 0;
    while (i < len)
    {
        dst[i] = src[i];
        i++;
    }
}

char      *ft_strjoin(char const *s1, char const *s2)
{
    char      *collab;
    size_t      s1_len;
    size_t      s2_len;

    if (!s1 || !s2)
        return (NULL);
    s1_len = ft_strlen(s1);
    s2_len = ft_strlen(s2);
    collab = malloc(s1_len + s2_len + 1);
    if (!collab)
        return (NULL);
    ft_strjoin_copy(collab, s1, s1_len);
    ft_strjoin_copy(collab + s1_len, s2, s2_len);
    collab[s1_len + s2_len] = '\0';
    return (collab);
}
```

ORNEK

```
ft_strjoin("foo", "bar"); // "foobar"
```

ft_strtrim (char const *s1, char const *set)

s1'in başından ve sonundan, set içinde geçen karakterleri temizler.

İki indeks tutuyor: `start` baştan ilerleyerek `set` içinde olmayan ilk karaktere kadar gidiyor, `end` sondan geriye giderek aynısını yapıyor. Her ikisi de `ft_strchr(set, s1[...])` ile "bu karakter set içinde mi" diye soruyor. Sonunda gerçek kırma işini kendisi yapmıyor -- zaten `ft_substr`'ın yaptığı şey tam olarak "şu aralığı al", o yüzden hesapladığı `start` ve `end - start` değerleriyle `ft_substr`'ı çağırıp işi ona devrediyor.

```
char      *ft_strtrim(char const *s1, char const *set)
{
    size_t      start;
    size_t      end;

    if (!s1)
        return (NULL);
    if (!set)
        return (ft_substr(s1, 0, ft_strlen(s1)));
    start = 0;
    end = ft_strlen(s1);
    while (start < end && ft_strchr(set, s1[start]))
        start++;
    while (end > start && ft_strchr(set, s1[end - 1]))
        end--;
    return (ft_substr(s1, start, end - start));
}
```

BENZETME

Bir kağıdın kenarlarındaki gereksiz boşlukları kırmak gibi düşün: sadece kenarlardan (baş ve son) kırıyorsun, ortadaki hiçbir şeye dokunmuyorsun.

ORNEK

```
ft_strtrim(" Merhaba ", " "); // "Merhaba"
ft_strtrim("xxHixx", "x");    // "Hi"
```

ft_split (char const *s, char c)

s string'ini c ayırıcı karakterine göre parçalara ayırıp, NULL ile biten bir string dizisi (char) döner.**

Bu, Part 2'nin en uzun fonksiyonu -- norm'un 25 satır sınırına sığması için tek bir dev fonksiyon yerine **4 küçük yardımcı fonksiyona** bölündü. Her birinin tek bir görevi var, sırayla nasıl bir araya geldiklerini anlatıyorum:

1. **ft_countwords** : Önce kaç kelime olacağını sayıyor. Neden önce sayıyoruz? Çünkü sonuç dizisini (char **result) malloc ederken kaç göze ihtiyacımız olduğunu bilmemiz lazım -- C'de diziler dinamik büyümmez, baştan doğru boyutu ayırmak zorundayız.
2. **ft_write_word** : Tek bir kelimeyi (bir sonraki ayırıcıya ya da string sonuna kadar) yeni bellekte kopyalayıp döner.
3. **ft_split_fill** : **ft_write_word** 'ü çağırıp sonucu **result** dizisindeki doğru göze (***slot**) yerleştiriyor, sonra i indeksini o kelimenin sonuna kadar ilerletiyor (ayırıcının kendisini atlamiyor -- onu ana döngü hallediyor).
4. **ft_split_free** : Bellek ayırma sırasında bir yerde hata olursa (malloc **NULL** dönerse), o ana kadar başarıyla ayrılmış kelimeleri tek tek serbest bırakıp diziyi de siliyor -- yarım kalmış, sızıntı yapan bir yapı bırakmamak için.

Ana **ft_split** fonksiyonu bu parçaları birleştiriyor: **s** üzerinde ilerlerken ayırıcıya denk gelirse sadece **i** 'yi artırıyor, değilse **ft_split_fill** 'i çağırıp bir kelime çıkartıyor ve **result[j++]** ile hem yerleştirip hem sıradaki göze geçiyor. En sonda **result[j] = NULL** ile diziyi sonlandırıyor -- bu NULL, kullanıcının döngüyle diziyi gezerken "ne zaman duracağını" bilmesini sağlıyor.

```
static int      ft_countwords(const char *s, char c)
{
    int          count;

    count = 0;
    while (*s)
    {
        if (*s != c)
        {
            count++;
            while (*s && *s != c)
                s++;
        }
        else
            s++;
    }
    return (count);
}

static char      *ft_write_word(const char *s, char c)
{
    char          *result;
    int           i;

    i = 0;
    while (s[i] && s[i] != c)
        i++;
    result = malloc(sizeof(char) * (i + 1));
    if (!result)
        return (NULL);
    i = 0;
    while (s[i] && s[i] != c)
    {
        result[i] = s[i];
        i++;
    }
    result[i] = '\0';
    return (result);
}

static void      ft_split_free(char **result, int j)
{
    while (j > 0)
        free(result[--j]);
    free(result);
}

static int      ft_split_fill(const char *s, int *i, char c, char **slot)
```

```

{
    *slot = ft_write_word(&s[*i], c);
    if (!*slot)
        return (-1);
    while (s[*i] && s[*i] != c)
        (*i)++;
    return (0);
}

char **ft_split(char const *s, char c)
{
    char **result;
    int i;
    int j;

    if (!s)
        return (NULL);
    i = 0;
    j = 0;
    result = malloc(sizeof(char *) * (ft_countwords(s, c) + 1));
    if (!result)
        return (NULL);
    while (s[i])
    {
        if (s[i] == c)
            i++;
        else if (ft_split_fill(s, &i, c, &result[j++]) == -1)
            return (ft_split_free(result, j), NULL);
    }
    result[j] = NULL;
    return (result);
}

```

BENZETME

Uzun bir cümleyi makasla kelime kelime kesip ayrı zarflara koyduğunu düşün. Önce kaç zarfa ihtiyacın olduğunu sayıyorsun (`ft_countwords`), sonra her kelimeyi tek tek kesip kendi zarfına koyuyorsun (`ft_write_word` + `ft_split_fill`), zarfların hepsini bir kutuya diziyorsun (`result`), ve kutunun sonuna "bitti" etiketi (`NULL`) yapııştırıyorsun.

ORNEK

```

ft_split("Bu bir test", ' ');
// result[0] = "Bu"
// result[1] = "bir"
// result[2] = "test"
// result[3] = NULL

```

ft_itoa (int n)

n sayısını yeni ayrılmış bir string'e çevirir ("Integer to ASCII").

İki aşamalı: önce `ft_find_size` kaç karaktere ihtiyaç olduğunu hesaplıyor (basamak sayısı, negatifse `-` için +1). Sonra asıl fonksiyon, string'i **sondan başa** dolduruyor: en sondaki basamak en son hesaplanabildiği için (`n % 10` ile), `str[--len]` ile index'i önce bir azaltıp oraya yazıyoruz, en başa kadar geri geri gidiyoruz. Negatifse en başa `'-'` konup sayı pozitifte çevriliyor. `num` her yerde `long` tutuluyor -- sebebini aşağıda anlatıyorum.

```
static size_t      ft_find_size(int n)
{
    size_t      len;
    long        num;

    num = n;
    len = 0;
    if (num <= 0)
    {
        len++;
        num = -num;
    }
    while (num > 0)
    {
        num /= 10;
        len++;
    }
    return (len);
}

char      *ft_itoa(int n)
{
    char      *str;
    int        len;
    long        num;

    num = n;
    len = ft_find_size(num);
    str = (char *)malloc(sizeof(char) * (len + 1));
    if (!str)
        return (NULL);
    str[len] = '\0';
    if (num == 0)
        str[0] = '0';
    if (num < 0)
    {
        str[0] = '-';
        num = -num;
    }
    while (num > 0)
    {
        str[--len] = (num % 10) + '0';
        num /= 10;
    }
    return (str);
}
```

ORNEK

```
ft_itoa(42);           // "42"
ft_itoa(-2147483648); // "-2147483648" (INT_MIN dahi dogru calisiyor)
```

DİKKAT INT_MIN (-2147483648) özel bir tuzak: bu sayının pozitifliği (2147483648), int'in sığdırabileceği en büyük değeri (2147483647) bir fazlasıyla aşıyor. "num = -num" işlemi int ile yapılsaydı taşardı (undefined behavior). num'ı long tutmak bu taşmayı engelliyor, çünkü long çok daha geniş bir aralığı sığdırıyor.

ft_strmapi (char const *s, char (*f)(unsigned int, char))

s'in her karakterine f fonksiyonunu uygulayıp SONUCLARI yeni bir string'de toplar.

f burada bir **fonksiyon pointer'ı** -- yani parametre olarak bizzat bir fonksiyon alıyoruz (indeks ve karakteri alıp yeni bir karakter döndüren herhangi bir fonksiyon). Orijinal s'e hiç dokunmuyoruz, tamamen yeni bir string dolduruyoruz: her indekste f(i, s[i]) 'nin sonucunu result[i] 'e yazıyoruz.

```
char      *ft_strmapi(char const *s, char (*f)(unsigned int, char))
{
    char      *result;
    size_t      i;
    size_t      len;

    len = ft_strlen(s);
    result = malloc(len + 1);
    if (!result)
        return (NULL);
    i = 0;
    while (i < len)
    {
        result[i] = f(i, s[i]);
        i++;
    }
    result[len] = '\0';
    return (result);
}
```

ORNEK

```
char to_upper(unsigned int i, char c) { (void)i; return (c - 32 * (c >= 'a' && c <= 'z')); }
ft_strmapi("merhaba", to_upper); // yeni string: "MERHABA", "merhaba" degismedi
```

ft_striteri (char *s, void (*f)(unsigned int, char*))

s'in her karakterine f fonksiyonunu uygular, ama **ORIJINAL** string'i yerinde değiştirir.

ft_strmap'i 'ye çok benziyor ama iki önemli fark var: **(1)** yeni string döndürmüyor, void -- yani hiçbir şey dönmüyor, direkt s 'i değiştiriyor. **(2)** f 'e karakterin kendisini değil, **adresini** (&s[i]) veriyoruz -- böylece f içeride *c = ... yaparak orijinal karakteri gerçekten değiştirebiliyor.

```
void ft_striteri(char *s, void (*f)(unsigned int, char*))
{
    unsigned int i;

    if (!s || !f)
        return ;
    i = 0;
    while (s[i] != '\0')
    {
        f(i, &s[i]);
        i++;
    }
}
```

ORNEK

```
void star_even(unsigned int i, char *c) { if (i % 2 == 0) *c = '*'; }
char buf[] = "hello";
ft_striteri(buf, star_even); // buf artık "*e*l*" (yerinde degisti)
```

ft_putchar_fd (char c, int fd)

Bir karakteri verilen dosya tanımlayıcısına (file descriptor) yazar.

Doğrudan write() sistem çağrısını kullanıyor. fd (file descriptor) "nereye yazılacağını" belirten bir sayı: 1 ekran (stdout), 2 hata çıktısı (stderr), ya da açık bir dosyanın numarası olabilir. write, tek karakteri yazmak için karakterin adresini ve "1 byte yaz" bilgisini istiyor.

```
void ft_putchar_fd(char c, int fd)
{
    write(fd, &c, 1);
}
```

ft_putstr_fd (char *s, int fd)

Bir string'i verilen fd'ye, karakter karakter yazar.

`NULL` kontrolünden sonra, `'\0'` 'a kadar her karakteri tek tek `write` ile yazıyor. Tek seferde tüm string'i yazmak yerine karakter karakter gitmek biraz yavaş ama norm'un izin verdiği araçlarla (kendi yazdığımız fonksiyonlarla) en sade çözüm bu.

```
void ft_putstr_fd(char *s, int fd)
{
    int i;

    if (!s)
        return ;
    i = 0;
    while (s[i] != '\0')
    {
        write(fd, &s[i], 1);
        i++;
    }
}
```

ft_putendl_fd (char *s, int fd)

Bir string'i, sonuna yeni satır (\n) ekleyerek fd'ye yazar.

`ft_putstr_fd` 'nin neredeyse aynısı, tek fark döngüden sonra ayrıca `write(fd, "\\n", 1)` ile bir yeni satır ekliyor olması. "endl" ismi de zaten "end line"dan geliyor -- "satırı bitir" demek.

```
void ft_putendl_fd(char *s, int fd)
{
    int i;

    if (!s)
        return ;
    i = 0;
    while (s[i])
    {
        write(fd, &s[i], 1);
        i++;
    }
    write(fd, "\\n", 1);
}
```

ft_putnbr_fd (int n, int fd)

Bir int'i (rakam rakam, sayı olarak) verilen fd'ye yazar.

Bu fonksiyon **özyinelemeli (recursive)** -- kendi kendini çağırıyor. Mantığı şöyle: bir sayıyı soldan sağa (en anlamlı basamaktan başlayarak) yazdırmak istiyoruz ama elimizdeki araç (`% 10`) bize en sağdaki basamağı veriyor. Çözüm: eğer sayıda birden fazla basamak varsa (`num > 9`), önce `num / 10` için kendimizi tekrar çağırıyoruz (yani "önce soldaki kısmı yazdır"), **sonra** son basamağı (`num % 10`) yazdırıyoruz. Tek basamak kaldıysa (`else` dalı) direkt yazdırıp geri dönüyoruz -- bu da özyinelemenin **taban durumu (base case)**, yani kendini çağırmayı durduran koşul.

```
void ft_putnbr_fd(int n, int fd)
{
    char c;
    long num;

    num = n;
    if (num < 0)
    {
        num = -num;
        write(fd, "-", 1);
    }
    if (num > 9)
    {
        ft_putnbr_fd(num / 10, fd);
        ft_putnbr_fd(num % 10, fd);
    }
    else
    {
        c = (num % 10) + '0';
        write(fd, &c, 1);
    }
}
```

ORNEK

```
ft_putnbr_fd(-12345, 1); // ekrana "-12345" yazar
ft_putnbr_fd(0, 1);     // ekrana "0" yazar
```

DIKKAT

Taban durumu else içine alınmazsa, bir sayı hem 'ilk yazdırma' (num/10 çağırısı içinde tek basamağa inince) hem de 'ikinci yazdırma' (else olmadan kod her zaman devam edip alttaki satırı da çalıştırır) olmak üzere son rakamı iki kez yazdırabilir. Bu projede gerçekten yaşanan bir hataydı.

Bonus: Zincirli Liste (Linked List) Fonksiyonları

Bu bölüm konsept olarak öncekilerden biraz farklı: artık string ya da sayı değil, kendi kurduğumuz bir **veri yapısı** (`t_list`) üzerinde çalışıyoruz. `libft.h` bölümünde tren benzetmesini yapmıştık -- her `t_list` düğümü bir vagon, `content` vagonun yükü, `next` bir sonraki vagona giden kanca. Bu 9 fonksiyon, o treni yönetmenin bütün yollarını kapsıyor: vagon eklemek, saymak, sona ulaşmak, vagon silmek, bütün treni dağıtmak, her vagonun yüküne bir işlem uygulamak.

BENZETME

Zincirli listeyi bir dizile (`array`) karşılaştırmak faydalı. Bir dizide bütün elemanlar bellekte yan yana, art arda durur -- `arr[3]` 'e direkt zıplayabilirsin. Zincirli listede ise elemanlar bellekte **rastgele yerlerde** durabilir, birbirlerine sadece `next` pointer'larıyla bağlıdırlar. Bunun bedeli: 4. elemana ulaşmak için 1, 2 ve 3'ten geçmen (yani listeyi baştan gezmen) gerekir, direkt zıplayamazsın. Ama avantajı: araya yeni bir vagon eklemek ya da bir vagonu çıkarmak çok ucuz -- sadece birkaç kancayı değiştirmen yeterli, bütün diziyi kaydırmak gerekmiyor.

ft_lstnew (void *content)

content'i taşıyan, tek başına duran yeni bir liste düğümü (node) oluşturur.

Bütün liste fonksiyonlarının temeli bu -- yeni bir vagon üretiyor. `malloc` ile bir `t_list` boyutunda yer ayırıyor, verilen `content` 'i içine koyuyor, `next` 'i `NULL` yapıyor (henüz kimseye bağlı değil, tek başına duruyor).

```
t_list      *ft_lstnew(void *content)
{
    t_list      *node;

    node = (t_list *)malloc(sizeof(t_list));
    if (!node)
        return (NULL);
    node->content = content;
    node->next = NULL;
    return (node);
}
```

ft_lstadd_front (t_list **lst, t_list *new)

new düğümünü listenin en BAŞINA ekler.

`lst` 'nin neden çift pointer (`t_list **`) olduğu burada netleşiyor: listenin başını ("head"ini) gerçekten değiştirmemiz gerekiyor. Sadece `t_list *lst` alsaydık, fonksiyon içinde `lst` 'yi değiştirmek sadece yerel bir kopyayı değiştirmiş olurdu, çağıran taraftaki gerçek liste değişmezdi. Çift pointer sayesinde "listenin başının adresi"ni alıyoruz, oradan gerçek head'i güncelleyebiliyoruz: önce yeni düğümün kancasını eski head'e bağlıyoruz (`new->next = *lst`), sonra head'i yeni düğüm yapıyoruz (`*lst = new`).

```
void      ft_lstadd_front(t_list **lst, t_list *new)
{
    if (!lst || !new)
        return ;
    new->next = *lst;
    *lst = new;
}
```

BENZETME

Trene en öne yeni bir vagon eklemek gibi: yeni vagonun arka kancasını eski ilk vagona takıyorsun, sonra "tren artık bu yeni vagonla başlıyor" diye ilan ediyorsun.

ft_lstsize (t_list *lst)

Listedeki toplam düğüm (eleman) sayısını döner.

Baştan sona, `next` pointer'larını takip ederek sayaç artırıyor. `lst` `NULL` olunca (son vagondan sonrası) döngü kendiliğinden bitiyor -- bu yüzden boş liste için doğal olarak `0` döner, ayrı bir kontrole gerek yok.

```
int ft_lstsize(t_list *lst)
{
    int i;

    i = 0;
    while (lst)
    {
        i++;
        lst = lst->next;
    }
    return (i);
}
```

ft_lstlast (t_list *lst)

Listenin SON düğümünü döner.

Dikkat edilmesi gereken ince nokta: döngü koşulu `lst->next` (bir sonrakine bakıyor), `lst` değil. Eğer koşul sadece `lst` olsaydı, döngü son düğümün de **ötesine** geçip `NULL` 'a kadar giderdi ve fonksiyon yanlışlıkla `NULL` dönerdi -- bu projede gerçekten yaşanan bir hataydı. Doğrusu: "bir sonraki varken ilerle", yani `next` 'i `NULL` olan düğümde dur, çünkü orası son vagon.

```
t_list *ft_lstlast(t_list *lst)
{
    if (!lst)
        return (NULL);
    while (lst->next)
        lst = lst->next;
    return (lst);
}
```

DIKKAT "while (lst)" ile "while (lst->next)" arasındaki fark küçük görünüyor ama sonucu tamamen değiştiriyor. Bu tarz "bir adım ileriye bakan" döngüler zincirli listelerde çok sık karşına çıkacak, iyi oturt.

ft_lstadd_back (t_list **lst, t_list *new)

new düğümünü listenin en SONUNA ekler.

İki durumu ayrı ele alıyor: liste zaten boşsa (`*lst == NULL`), yeni düğüm direkt listenin kendisi oluyor. Doluysa, sona kadar yürüyüp (`while (last->next != NULL)` -- burada da `ft_lstlast` 'taki aynı "bir adım ileri bakma" mantığı var) son düğümün kancasını yeni düğüme bağlıyor.

```
void ft_lstadd_back(t_list **lst, t_list *new)
{
    t_list *last;

    if (!lst || !new)
        return ;
    if (*lst == NULL)
    {
        *lst = new;
        return ;
    }
    last = *lst;
    while (last->next != NULL)
        last = last->next;
    last->next = new;
}
```

ft_lstdelone (t_list *lst, void (*del)(void*))

TEK BİR düğümü, içeriğiyle (content) birlikte serbest bırakır.

Burada neden ayrı bir `del` fonksiyon pointer'ı alındığına dikkat et: `content` tipi `void*`, yani içinde ne olduğunu `ft_lstdelone` bilmiyor -- bir `int*` mi, bir `struct` mı, iç içe geçmiş bir yapı mı, sadece kullanıcı bilir. O yüzden içeriği nasıl temizleyeceğini (belki içinde başka malloc'lar bile var) kullanıcı bir fonksiyon olarak veriyor, biz sadece onu çağırıp (`del(lst->content)`) sonra düğümün kendisini (`free(lst)`) siliyoruz. **Not:** bu fonksiyon `lst->next` 'e hiç dokunmuyor -- listenin geri kalanını bağlamak/koparmak çağırının işi, bu sadece tek vagonu söküyor.

```
void ft_lstdelone(t_list *lst, void (*del)(void*))
{
    if (!lst || !del)
        return ;
    del(lst->content);
    free(lst);
}
```

ft_lstclear (t_list **lst, void (*del)(void*))

Listenin TAMAMINI, baştan sona her düğümü içeriğiyle birlikte siler.

En kritik satır: `temp = (*lst)->next;` -- bu satır, `ft_lstdeleteone` ile şu anki düğümü **silmeden önce** bir sonraki düğümün adresini bir kenara not ediyor. Neden gerekli? Çünkü `ft_lstdeleteone` düğümü `free` 'ledikten sonra o belleğe erişmek tanımsız davranış (kullanılmış belleği okumak) olur -- eğer "sıradaki" adresi önceden kaydetmeseydik, düğümü sildikten sonra bir sonrakine nasıl gideceğimizi bilemezdik. Silip sonra `*lst = temp` ile bir adım ilerliyoruz, liste bitene kadar tekrarlıyoruz.

```
void ft_lstclear(t_list **lst, void (*del)(void*))
{
    t_list *temp;

    if (!lst || !*lst || !del)
        return ;
    while (*lst)
    {
        temp = (*lst)->next;
        ft_lstdeleteone(*lst, del);
        *lst = temp;
    }
    *lst = NULL;
}
```

BENZETME Bir tren garajını tamamen boşaltmak gibi: her vagonu sökmeden önce "bir sonraki vagon hangisiydi" diye not alıyorsun, vagonu hurdaya çıkarıyorsun, sonra notundaki vagona geçip aynısını tekrarlıyorsun.

ft_lstiter (t_list *lst, void (*f)(void*))

Listedeki HER düğümün content'ine f fonksiyonunu uygular (listeyi değiştirmez).

En basit bonus fonksiyonlarından biri: baştan sona gezip her adımda `f(lst->content)` çağırıyor. Genelde yazdırma ya da içerik üzerinde yan etkili (side-effect) bir işlem için kullanılır -- yeni bir liste üretmez, sadece var olanı gezip bir şey yapar.

```
void ft_lstiter(t_list *lst, void (*f)(void *))
{
    if (!lst || !f)
        return ;
    while (lst)
    {
        f(lst->content);
        lst = lst->next;
    }
}
```

ORNEK

```
void print_int void *c) { printf("%d ", *(int *)c); }
ft_lstiter(liste, print_int); // liste: 2 1 3 şeklinde yazdırır
```

```
ft_lstmap (t_list *lst, void *(*f)(void *), void (*del)(void *))
```

Her düğümün content'ine f uygulayıp SONUÇLARDAN yepyeni bir liste inşa eder.

ft_lstiter'dan farkı: o sadece geziyordu, bu **yeni bir liste kuruyor**. Her adımda f(lst->content) ile yeni bir içerik üretiyor, bunu ft_lstnew ile bir düğüme sarıyor ve ft_lstadd_back ile inşa ettiğimiz yeni listenin sonuna ekliyor -- yani bu fonksiyon, öğrendiğin diğer üç bonus fonksiyonunu (lstnew, lstadd_back, ve hata durumunda lstclear) birleştirerek çalışıyor.

Hata durumu da düşünülmüş: eğer bir düğüm için ft_lstnew bellek ayıramazsa (NULL dönerse), o ana kadar oluşturduğumuz new_list'i ft_lstclear ile tamamen temizleyip NULL dönüyoruz -- yarım kalmış, sızıntı yapan bir liste bırakmıyoruz.

```
t_list      *ft_lstmap(t_list *lst, void *(*f)(void *), void (*del)(void *))
{
    t_list      *new_list;
    t_list      *new_node;
    void        *content;

    if (!lst || !f || !del)
        return (NULL);
    new_list = NULL;
    while (lst)
    {
        content = f(lst->content);
        new_node = ft_lstnew(content);
        if (!new_node)
        {
            del(content);
            ft_lstclear(&new_list, del);
            return (NULL);
        }
        ft_lstadd_back(&new_list, new_node);
        lst = lst->next;
    }
    return (new_list);
}
```

ORNEK

```
void *double_int(void *c) { int *v = malloc(sizeof(int)); *v = *(int *)c * 2; return (v); }
ft_lstmap(liste, double_int, free); // her elemani 2 ile carpilmis YENI bir liste doner
```

test_libft.c: Bunları Nasıl Doğruladık

Kütüphanenin yanında bir de `test_libft.c` var, zorunlu değil ama savunmada (defense) işine yarasın diye tutuyoruz. Kod satırlarını tek tek anlatmayacağım (yukarıda her fonksiyonun kendi örneği zaten vardı), ama nasıl kurgulandığını anlamak faydalı.

check() ve section(): küçük bir mini test altyapısı

`check(sart, isim)` fonksiyonu basit: verilen şart doğruysa yeşil `[OK]`, yanlışsa kırmızı `[FAIL]` yazdırıp bir sayaç (`g_pass / g_fail`) artırıyor. `section(baslik)` ise her kategoriye geçerken bir önceki kategorinin özetini (kaç/kaç geçti) yazdırıp yeni başlığı açıyor. Bu ikisi sayesinde tek tek `printf` yazmak yerine, her test satırı tek satırlık okunabilir bir `check(...)` çağrısına dönüşüyor.

Neden bazı testler C'nin kendi fonksiyonlarıyla karşılaştırıyor

Mesela `ft_memmove` testinde önce sistemin gerçek `memmove` 'unu çalıştırıp sonucu bir referans dizisine alıyoruz, sonra aynı girdiyle `ft_memmove` 'u çalıştırıp ikisini `memcmp` ile karşılaştırıyoruz. Bu, "doğru cevap neydi" diye elle hesaplamak yerine, C'nin kendi güvenilir implementasyonunu **referans** olarak kullanmanın pratik bir yolu.

Neden bazı fonksiyonlar görsel doğrulama istiyor

`ft_putnbr_fd` gibi fonksiyonlar bir değer **döndürmüyor**, direkt ekrana yazıyor -- yani otomatik `check()` ile "doğru mu" diye ölçmek yerine, "Beklenen" ve "Gerçek" satırlarını yan yana yazdırıp gözle karşılaştırma yapıyoruz. Küçük ama önemli bir hatayı (son basamağın iki kere yazdırılması) gerçekten bu şekilde, gözle fark ederek yakaladık.

Sonuç: 61/61

Şu an kütüphanedeki her fonksiyon en az bir, çoğu birden fazla senaryoyla test ediliyor (normal durum, sınır durumu, hatalı girdi durumu) ve toplamda 61 test `[OK]` veriyor. Bu, "muhtemelen doğru" ile "gerçekten denedim, çalıştığını gördüm" arasındaki farkı kapatıyor.

Kapanış

Artık bildiklerin

Bu dökümanı baştan sona okuduysan artık şunları biliyorsun: bir .c dosyasının nasıl .o'ya, .o'ların nasıl libft.a'ya dönüştüğünü; Makefile'daki her satırın neden orada olduğunu; libft.h'ın neden bir "kimlik kartı" gibi çalıştığını; ve kütüphanedeki 43 fonksiyonun her birinin -- basit bir isalpha'dan en karmaşık split ve lstmap'e kadar -- içeride tam olarak ne yaptığını.

Bundan sonra push_swap, minishell gibi projelerde `ft_split` ya da `ft_lstnew` çağırdığında, artık kara kutu bir fonksiyon çağırmıyorsun -- içeride ne olduğunu, neden o şekilde yazıldığını biliyorsun. Zaman zaman kodun kendisine geri dönüp okumak, bu dökümanı tekrar açmaktan daha iyi bir pekiştirme olacaktır.